

Pipeline hồi quy với tiền xử lý và giảm chiều PCA

Dự báo chỉ số tiến triển bệnh trên bộ dữ liệu Diabetes

Sinh viên Lương Đức Mạnh

Mã học viên B26CHKH068

Lớp M26CQKH03-B

Năm học 2026 - 2027

1. Bài toán và bộ dữ liệu

Đề Bài tập 2 yêu cầu chọn dữ liệu có target liên tục, xây dựng chương trình dự báo, so sánh baseline với các mô hình hồi quy và kiểm tra tác động của PCA. Bài này sử dụng Diabetes tích hợp trong scikit-learn, có 442 mẫu và 10 biến đầu vào [1]. Target là chỉ số định lượng về tiến triển bệnh sau một năm; dù dữ liệu ghi bằng số nguyên, đây là đại lượng cần hồi quy, không phải nhãn phân lớp.

Bảng 1. Biến đầu vào và target

Biến	Ý nghĩa trong dữ liệu
age	Tuổi
sex	Biến giới tính đã mã hóa trong bộ dữ liệu
bmi	Chỉ số khối cơ thể
bp	Huyết áp trung bình
s1, s2, s3	Các chỉ số cholesterol toàn phần, LDL và HDL
s4	Tỷ số cholesterol toàn phần trên HDL
s5	Biến ltg theo mô tả dữ liệu; giữ nguyên giá trị nguồn
s6	Chỉ số đường huyết
target	Chỉ số định lượng tiến triển bệnh sau một năm

Chương trình gọi `load_diabetes(scaled=False)` để nhận các biến chưa chuẩn hóa bởi hàm tải dữ liệu. Điều này cho phép `StandardScaler` học trung bình và độ lệch chuẩn chỉ từ Train hoặc phần huấn luyện của từng fold. Không dùng chế độ `scaled=True` mặc định để tránh đưa thống kê của cả bộ dữ liệu vào bước tiền xử lý [1].

Kiểm tra dữ liệu cho thấy không có giá trị thiếu, không có giá trị vô hạn và không có hai hàng X trùng hoàn toàn. Target nằm trong khoảng 25 đến 346. Không loại ngoại lệ hoặc chọn đặc trưng dựa trên kết quả Test. Biến sex được giữ dưới dạng mã số đã cung cấp; bài không suy diễn ý nghĩa của từng mã.

sample_id trong các tệp CSV là chỉ số hàng gốc bắt đầu từ 0, chỉ dùng để đối chiếu chia tập. Đây không phải đặc trưng của mô hình hay định danh bệnh nhân. Tệp `train.csv` và `test.csv` lưu đúng 10 biến theo thứ tự age, sex, bmi, bp, s1, s2, s3, s4, s5, s6 cùng target.

2. Pipeline và thiết kế đánh giá

2.1. Tiền xử lý bên trong mô hình

Pipeline thực hiện theo thứ tự: điền thiếu bằng trung vị → chuẩn hóa StandardScaler → PCA nếu được bật → mô hình hồi quy. Bộ dữ liệu hiện không thiếu, nên SimpleImputer không làm thay đổi các giá trị; bước này giữ cấu trúc xử lý nhất quán. Target được giữ nguyên thang đo.

$$z = \frac{x - \mu}{\sigma}$$

μ và σ được ước lượng riêng cho từng cột trên phần dữ liệu đang dùng để fit. Validation/Test chỉ được transform bằng các tham số đã học. PCA dùng SVD đầy đủ và whiten=False. PCA tự căn giữa nhưng không tự chuẩn hóa thang đo từng biến, nên StandardScaler được đặt trước PCA [2, 3].

2.2. Tách Test trước khi chọn mô hình

Tách ngẫu nhiên 80% Train và 20% Test với random_state = 68, thu được 353 và 89 mẫu. Test không tham gia chọn siêu tham số, số thành phần PCA hoặc họ mô hình. Baseline cũng chỉ học giá trị trung bình target của phần huấn luyện.

Bảng 2. Vai trò của các vòng đánh giá

Bước	Thiết lập	Dữ liệu dùng
Tách Test	test_size = 0,2; seed = 68	442 mẫu ban đầu
CV ngoài	KFold 5; shuffle; seed = 70	353 mẫu Train
CV trong	KFold 3; shuffle; seed = 69	Phần train của từng fold ngoài
Fit cuối	Tìm tham số lại bằng CV trong	Toàn bộ 353 mẫu Train
Đánh giá cuối	Một lượt sau khi chốt họ mô hình	89 mẫu Test

Ở mỗi fold ngoài, GridSearchCV chọn cấu hình có RMSE CV trong thấp nhất rồi fit lại trên phần train của fold ngoài. Mô hình đó dự báo phần validation ngoài để tính MAE, RMSE và R^2 . Các phương án dùng chung năm cách chia ngoài, giúp so sánh trên cùng tập validation. Pipeline bảo đảm imputer, scaler và PCA đều được fit lại trong từng lượt [2, 4].

Điểm CV ngoài đánh giá quy trình tìm tham số trong từng họ mô hình. Vì còn dùng các điểm này để chọn giữa sáu họ, điểm của họ thắng vẫn có thể chịu thiên lệch lựa chọn; tập Test giữ riêng là đánh giá cuối. Bài không coi trung bình $\pm$ SD của năm fold là khoảng tin cậy hay kiểm định khác biệt có ý nghĩa thống kê.

3. Mô hình và tiêu chí lựa chọn

Bảng 3. Các phương án và không gian tìm kiếm

Tên	Mô hình và siêu tham số
Mean	DummyRegressor(strategy="mean")
Linear	LinearRegression có hệ số chặn
Ridge	$\alpha \in \{0,1; 1; 10; 100\}$; solver="svd"
Ridge_PCA	Như Ridge; $k \in \{2; 4; 6; 8; 0,95\}$
SVR	RBF; $C \in \{10; 100; 1000\}$; $\gamma \in \{0,01; 0,1\}$
SVR_PCA	Như SVR; $k \in \{2; 4; 6; 8; 0,95\}$

SVR cố định epsilon = 0,1 và không chuẩn hóa target. Giá trị $k = 0,95$ có nghĩa là chọn đủ số thành phần để giải thích trên 95% phương sai của phần dữ liệu dùng để fit PCA, không phải giữ 0,95 thành phần [3]. Các mô hình PCA có thể chọn k khác nhau giữa các fold.

Linear tối thiểu hóa tổng bình phương sai số. Ridge bổ sung phạt L2 cho hệ số, giúp kiểm soát hệ số lớn khi các biến tương quan. SVR với kernel RBF mô tả quan hệ phi tuyến; C và γ điều chỉnh mức phạt và phạm vi tác động của kernel. Các họ mô hình chỉ được so sánh trong phạm vi lưới tham số đã nêu.

3.1. Các thước đo

$$MAE = \frac{\sum |y - \hat{y}|}{n}$$

$$RMSE = \sqrt{\frac{\sum (y - \hat{y})^2}{n}}$$

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

Các tổng chạy trên n mẫu của tập đang đánh giá; $\bar{y}$ là trung bình target của chính tập đó. MAE và RMSE cùng đơn vị với target, càng thấp càng tốt; RMSE nhạy hơn với sai số lớn. R^2 càng cao càng tốt và có thể âm khi dự báo kém hơn dùng trung bình của tập đánh giá. R^2 không phải phần trăm dự đoán chính xác.

3.2. Quy tắc chốt họ mô hình

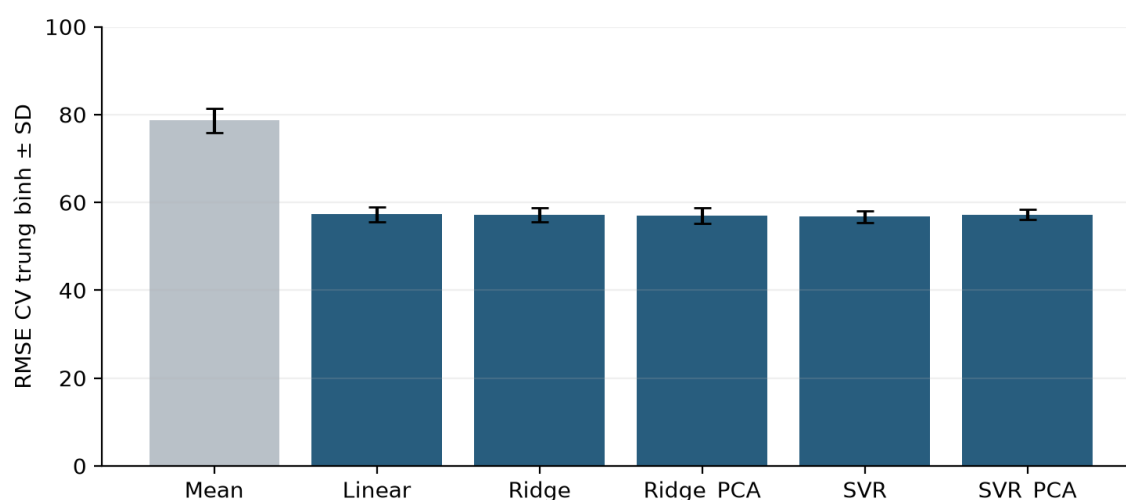
Tiêu chí chính là RMSE CV ngoài trung bình nhỏ nhất. Nếu bằng nhau, lần lượt xét MAE trung bình nhỏ hơn, SD RMSE nhỏ hơn, rồi R^2 trung bình lớn hơn. Báo cáo vẫn đối chiếu tất cả thước đo để nhận diện đánh đổi. SD được tính theo công thức độ lệch chuẩn mẫu với ddof = 1 trên năm fold ngoài.

4. Kết quả cross validation

Bảng 4. Trung bình $\pm$ SD trên 5 fold CV ngoài

Mô hình	MAE	RMSE	R ²
Mean	67,01 $\pm$ 2,38	78,66 $\pm$ 2,77	-0,0213 $\pm$ 0,0314
Linear	47,01 $\pm$ 2,45	57,27 $\pm$ 1,69	0,4562 $\pm$ 0,0556
Ridge	46,94 $\pm$ 2,67	57,14 $\pm$ 1,55	0,4593 $\pm$ 0,0487
Ridge_PCA	46,93 $\pm$ 2,74	57,02 $\pm$ 1,73	0,4614 $\pm$ 0,0485
SVR	46,57 $\pm$ 2,55	56,77 $\pm$ 1,36	0,4667 $\pm$ 0,0422
SVR_PCA	46,93 $\pm$ 2,63	57,22 $\pm$ 1,17	0,4581 $\pm$ 0,0407

Hình 1. RMSE CV ngoài với thanh sai số biểu diễn SD



Các mô hình hồi quy giảm rõ sai số so với baseline Mean: RMSE khoảng 56,77–57,27 so với 78,66. SVR có RMSE trung bình 56,77, MAE trung bình 46,57 và R² trung bình 0,4667, tốt nhất theo các trung bình này nên được chọn trước khi xem Test.

SVR_PCA có SD RMSE nhỏ nhất trong các phương án hồi quy là 1,17, nhưng RMSE trung bình cao hơn SVR khoảng 0,46. Ridge_PCA chỉ cải thiện RMSE trung bình khoảng 0,11 so với Ridge. Các khoảng biến động chồng lấp, vì vậy không có căn cứ từ năm fold để khẳng định SVR vượt trội có ý nghĩa thống kê.

Baseline Mean có R² CV trung bình âm vì giá trị trung bình học từ phần train của fold không nhất thiết trùng trung bình của validation. Đây là hành vi hợp lệ. Toàn bộ điểm từng fold, tham số chọn trong mỗi fold và dự báo ngoài fold được lưu trong `cv_folds.csv` và `oof_predictions.csv`.

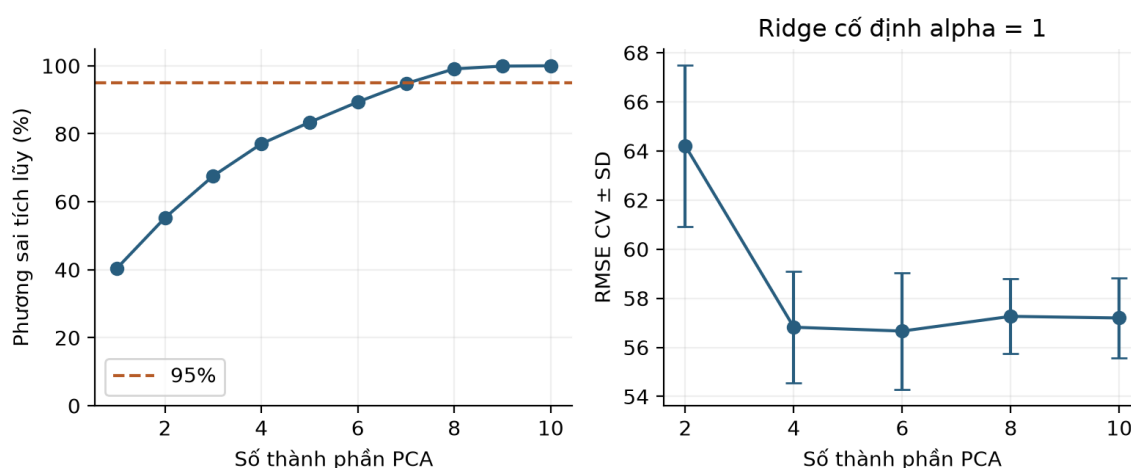
5. PCA thay đổi kết quả như thế nào

PCA tìm các hướng có phương sai lớn trong X đã chuẩn hóa và chiếu dữ liệu lên k hướng đầu. Với W chứa k vector trực chuẩn, phép chiếu là $Z = XW$. PCA không dùng target khi tìm hướng, nên thành phần có phương sai nhỏ vẫn có thể chứa tín hiệu hữu ích cho hồi quy.

Bảng 5. Khảo sát riêng PCA với Ridge cố định $\alpha = 1$

k	Phương sai Train tích lũy	RMSE CV trung bình $\pm$ SD
2	55,26%	64,21 $\pm$ 3,29
4	77,05%	56,82 $\pm$ 2,26
6	89,35%	56,66 $\pm$ 2,38
8	99,10%	57,26 $\pm$ 1,53
10	100,00%	57,20 $\pm$ 1,63

Hình 2. Phương sai giữ lại và sai số khi thay số thành phần PCA



Trên toàn bộ Train, 7 thành phần giữ 94,78% phương sai, chưa đạt 95%; 8 thành phần giữ 99,10% nên đạt điều kiện. Hai thành phần chỉ giữ 55,26% phương sai và cho RMSE CV 64,21. Trong khảo sát cố định $\alpha = 1$, $k = 6$ có RMSE CV thấp nhất là 56,66 dù chỉ giữ 89,35% phương sai.

Bảng 5 là khảo sát có kiểm soát: α cố định, chỉ thay k ; scaler và PCA vẫn fit lại theo từng fold. Cột phương sai là mô tả PCA fit trên toàn bộ Train, không phải thống kê được dùng trước cho các fold. Không dùng khảo sát này để đổi lại họ mô hình đã chọn hoặc đổi chiều trực tiếp điểm nhỏ nhất với quy trình nested CV ở Bảng 4.

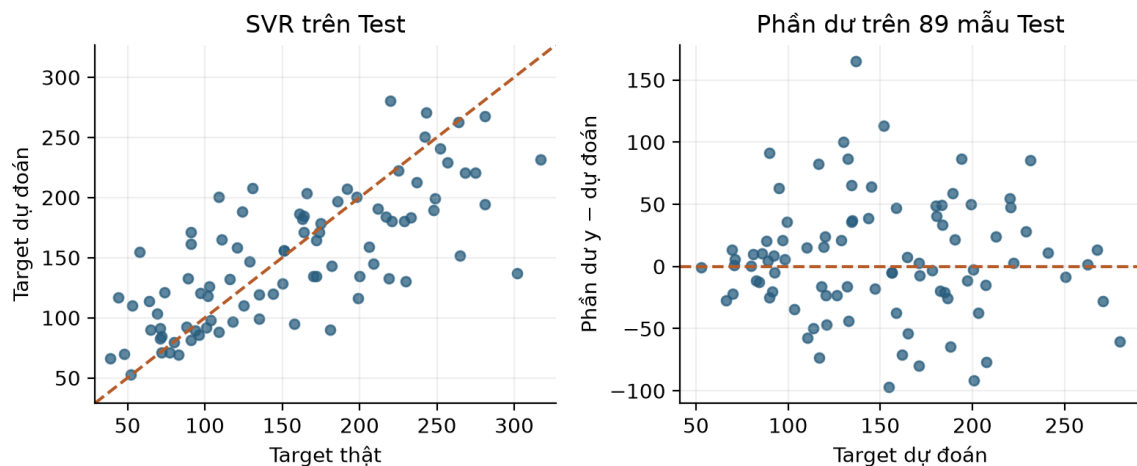
Trong tìm kiếm tham số cuối trên Train, Ridge_PCA chọn $k = 6$, $\alpha = 10$; SVR_PCA chọn $k = 4$, $C = 100$, $\gamma = 0,01$. Kiểm tra bổ sung cho thấy Ridge $\alpha = 1$ dùng đủ 10 thành phần PCA và Ridge không PCA cho dự báo Test lệch tối đa khoảng $6,82 \times 10^{-13}$. Điều này phù hợp với PCA đầy đủ chỉ quay hệ trục khi không whitening.

6. Đánh giá trên tập Test giữ riêng

Bảng 6. Kết quả trên 89 mẫu Test sau khi fit lại trên Train

Mô hình	MAE	RMSE	R ²
Mean	61,04	71,16	-0,0045
Linear	36,43	47,70	0,5486
Ridge	36,43	47,72	0,5483
Ridge_PCA	37,45	48,46	0,5343
SVR	36,17	47,84	0,5461
SVR_PCA	38,33	50,08	0,5026

Hình 3. Dự báo và phần dư của SVR trên tập Test



SVR được chọn từ CV đạt MAE 36,17, RMSE 47,84 và R² 0,5461. So với Mean, RMSE Test giảm khoảng 32,78%. Linear có RMSE Test thấp hơn SVR khoảng 0,13, nhưng đây là kết quả quan sát sau cùng; bài giữ lựa chọn SVR theo quy tắc CV đã chốt.

PCA làm RMSE Test của Ridge tăng từ 47,72 lên 48,46 và của SVR tăng từ 47,84 lên 50,08. Do các pipeline được tìm tham số riêng, đây là so sánh các quy trình đã tối ưu trong lưới, không tách riêng tác động của k. Khảo sát cố định α ở Bảng 5 bổ sung góc nhìn đó.

Phần dư được định nghĩa là y thật trừ dự đoán. Độ phân tán còn lớn cho thấy mô hình chưa dự báo chính xác từng mẫu. Biểu đồ dùng để mô tả sai số, không được dùng để chỉnh tiếp tham số rồi công bố lại cùng điểm Test. Test thấp hơn CV có thể do khác biệt của lần chia hữu hạn; không đủ để kết luận mô hình luôn đạt mức sai số này.

7. Ví dụ dự báo và giới hạn

7.1. Một quan sát chưa dùng để fit

Mẫu có `sample_id = 400` thuộc Test, được chọn theo vị trí đầu tiên trong tập Test của lần chia cố định, không được chọn vì có sai số nhỏ. Các giá trị đầu vào dưới đây giữ đúng thứ tự mà chương trình dự báo yêu cầu.

Bảng 7. Đầu vào của mẫu minh họa

age	sex	bmi	bp	s1	s2	s3	s4	s5	s6
42.0	1.0	30.6	121.0	176.0	92.8	69.0	3.0	4.2627	89.0

SVR được fit trên 353 mẫu Train dự báo target khoảng 178,26; target thật của mẫu là 175,00, sai số tuyệt đối khoảng 3,26. Đây chỉ là một ví dụ về luồng dự báo và không đại diện cho sai số trung bình của 89 mẫu Test.

Tập `predict_new.py` chỉ đọc Train, họ mô hình và cấu hình cuối đã chọn để fit lại rồi dự báo 10 giá trị đầu vào. Target của mẫu mới không được truyền vào mô hình. Lệnh python `predict_new.py` chạy ví dụ mặc định; cũng có thể cung cấp 10 số sau tên tệp.

7.2. Giới hạn

Diabetes là bộ dữ liệu nhỏ, chỉ có 89 mẫu Test trong lần chia này. Kết quả phụ thuộc vào cách chia và lưới tham số; một lần 5-fold CV không bao quát mọi biến động có thể có. SD giữa fold là chỉ báo độ ổn định mô tả, các fold không phải các thí nghiệm hoàn toàn độc lập.

Tách ngẫu nhiên giả định các hàng phù hợp để đánh giá theo cách chia này. Dữ liệu cung cấp không có thông tin nhóm hoặc thời gian đủ để kiểm chứng khái quát sang bệnh viện khác hay quần thể khác. Các chỉ số này không đánh giá hiệu quả sử dụng trong thực hành y khoa.

PCA giảm số chiều nhưng làm các thành phần khó diễn giải theo từng biến gốc. Giữ 95% phương sai không bảo đảm giữ 95% thông tin về target. Bài không thực hiện lựa chọn đặc trưng bằng target, không thử toàn bộ mô hình có thể có và không kiểm định ưu thế thống kê giữa các phương án gần nhau.

7.3. Kết luận

Pipeline đã đáp ứng yêu cầu tiền xử lý, baseline, so sánh hồi quy và đánh giá PCA bằng MAE, RMSE, R^2 cùng biến động CV. SVR không PCA được chọn theo RMSE CV trung bình thấp nhất; PCA hữu ích để khảo sát giảm chiều nhưng chưa cải thiện kết quả Test trong hai cặp mô hình của thực nghiệm. Quyết định giữ hoặc bỏ PCA cần dựa vào đánh giá dự báo, thay vì chỉ dựa vào tỷ lệ phương sai.

8. Tái tạo kết quả và tài liệu tham khảo

8.1. Cách chạy

Môi trường đã kiểm chứng: Python 3.12.14, NumPy 2.5.3, SciPy 1.18.1, scikit-learn 1.9.0 và matplotlib 3.11.1. Tập requirements.txt cố định phiên bản thư viện. Dữ liệu Diabetes đi kèm scikit-learn nên không cần tải dữ liệu từ Internet khi chạy sau khi cài thư viện.

```
python -m pip install -r requirements.txt
```

```
python regression_pca.py
```

```
python plot_results.py
```

```
python predict_new.py
```

8.2. Tập kết quả và kiểm tra

Thư mục ket_qua chứa Train/Test, danh sách fold, điểm và dự báo CV, lựa chọn mô hình, siêu tham số cuối, dự báo Test, khảo sát PCA và ba hình. results.json ghi phiên bản cùng SHA-256 của dữ liệu để đối chiếu. Mã regression_pca.py đầy đủ được in ở Phụ lục A; mã vẽ và dự báo nằm trong gói đi kèm.

Các kiểm tra tự động gồm: số mẫu và kích thước dữ liệu, Train/Test không giao nhau, dữ liệu hữu hạn, median và mean của preprocessing cuối khớp Train, tính lại MAE/RMSE/R² bằng NumPy và kiểm tra Ridge tương đương khi PCA giữ đủ chiều. Các kiểm tra đã đạt trong lần chạy tạo báo cáo.

Muốn khảo sát khác, thay dữ liệu, lưới hoặc seed rồi tạo một bộ kết quả mới. Không được chọn cấu hình dựa trên Test hiện tại và tiếp tục xem đó là đánh giá độc lập. Chênh lệch rất nhỏ ở chữ số cuối có thể xuất hiện giữa môi trường đại số tuyến tính khác nhau.

Tài liệu tham khảo

[1] scikit-learn. load_diabetes và mô tả Diabetes đi kèm gói dữ liệu. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html

[2] scikit-learn. Common pitfalls and recommended practices, mục Data leakage. https://scikit-learn.org/stable/common_pitfalls.html

[3] scikit-learn. PCA API reference. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

[4] scikit-learn. Nested versus non-nested cross-validation. https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html

Phụ lục A Mã thí nghiệm đầy đủ

Tập regression_pca.py. Các phần dưới đây nối tiếp nhau, không lược bỏ mã. Có thể chạy trực tiếp tệp đi kèm để tránh lỗi sao chép từ PDF.

```
"""Bai tap 2: train/test, nested CV va tac dong cua PCA."""
import csv
import hashlib
import json
import platform
from pathlib import Path
import numpy as np
import scipy
import sklearn
from sklearn.base import clone
from sklearn.datasets import load_diabetes
from sklearn.decomposition import PCA
from sklearn.dummy import DummyRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, KFold, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR

BASE = Path(__file__).resolve().parent
OUT = BASE / "ket_qua"
SEED = 68

def save_csv(name, rows):
    with (OUT / name).open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)

def metrics(y, pred):
    return dict(MAE=float(mean_absolute_error(y, pred)),
                RMSE=float(np.sqrt(mean_squared_error(y, pred))),
                R2=float(r2_score(y, pred)))

def pipeline(reg, k=None):
    steps = [("impute", SimpleImputer(strategy="median")),
             ("scale", StandardScaler())]
```

Phụ lục A Mã thí nghiệm phần 2

```
if k is not None:
    steps.append(("pca", PCA(n_components=k, svd_solver="full")))
return Pipeline(steps + [("reg", reg)])

def candidates():
    a = {"reg__alpha": [0.1, 1, 10, 100]}
    s = {"reg__C": [10, 100, 1000], "reg__gamma": [0.01, 0.1]}
    ks = {"pca__n_components": [2, 4, 6, 8, 0.95]}
    return {
        "Mean": (pipeline(DummyRegressor(strategy="mean")), {}),
        "Linear": (pipeline(LinearRegression()), {}),
        "Ridge": (pipeline(Ridge(solver="svd")), a),
        "Ridge_PCA": (pipeline(Ridge(solver="svd"), 4), a | ks),
        "SVR": (pipeline(SVR(kernel="rbf", epsilon=0.1)), s),
        "SVR_PCA": (pipeline(SVR(kernel="rbf", epsilon=0.1), 4), s | ks)
    }

def tune(estimator, grid, x, y):
    if not grid:
        return clone(estimator).fit(x, y), {}
    search = GridSearchCV(
        clone(estimator), grid, scoring="neg_root_mean_squared_error",
        cv=KFold(3, shuffle=True, random_state=SEED + 1),
        n_jobs=1, error_score="raise", refit=True)
    search.fit(x, y)
    return search.best_estimator_, search.best_params_

def main():
    OUT.mkdir(exist_ok=True)
    data = load_diabetes(scaled=False)
    x, y = data.data, data.target
    ids = np.arange(len(y))
    train, test = train_test_split(ids, test_size=0.2, random_state=SEED)
    assert x.shape == (442, 10) and len(train) == 353 and len(test) == 89
    assert not set(train) & set(test)
    assert np.isfinite(x).all() and np.isfinite(y).all()
    xtr, ytr, xte, yte = x[train], y[train], x[test], y[test]
    for name, ix in [("train.csv", train), ("test.csv", test)]:
        save_csv(name, [dict(sample_id=int(i), **dict(zip(data.feature_names,
```

Phụ lục A Mã thí nghiệm phần 3

```
x[i].tolist()))), target=float(y[i])) for i in ix])
outer = list(KFold(5, shuffle=True, random_state=SEED + 2).split(xtr))
save_csv("outer_folds.csv", [dict(sample_id=int(train[i]), fold=f + 1)
    for f, (_, va) in enumerate(outer) for i in va])
definitions = candidates()
fold_rows, summary, oof_rows = [], [], []
for name, (estimator, grid) in definitions.items():
    model_rows = []
    for fold, (tr, va) in enumerate(outer, 1):
        fitted, params = tune(estimator, grid, xtr[tr], ytr[tr])
        pred = fitted.predict(xtr[va])
        row = dict(model=name, fold=fold, **metrics(ytr[va], pred))
        row["parameters"] = json.dumps(params, sort_keys=True)
        model_rows.append(row)
        oof_rows.extend(dict(model=name, sample_id=int(train[i]),
            target=float(ytr[i]), prediction=float(p))
            for i, p in zip(va, pred))
    fold_rows.extend(model_rows)
    summary.append(dict(model=name, **{
        f"{m}_{stat}": float(fn([r[m] for r in model_rows]))
        for m in ["MAE", "RMSE", "R2"]
        for stat, fn in [("mean", np.mean),
            ("std", lambda v: np.std(v, ddof=1))]}))
    print(name, "CV RMSE", summary[-1]["RMSE_mean"], flush=True)
# Chọn mô hình từ CV, trước khi tính kết quả Test.
winner = min(summary, key=lambda r: (r["RMSE_mean"], r["MAE_mean"],
    r["RMSE_std"], -r["R2_mean"]))["model"]
save_csv("cv_folds.csv", fold_rows)
save_csv("cv_summary.csv", summary)
save_csv("oof_predictions.csv", oof_rows)
(OUT / "selection.json").write_text(json.dumps(
    dict(winner=winner, criterion="CV RMSE mean; tie MAE, RMSE SD, R2"),
    indent=2), encoding="utf-8")
final_rows, all_preds, final_params = [], {}, {}
for name, (estimator, grid) in definitions.items():
    fitted, params = tune(estimator, grid, xtr, ytr)
    pred = fitted.predict(xte)
    all_preds[name] = pred.tolist()
    final_params[name] = params
    final_rows.append(dict(model=name, **metrics(yte, pred)))
if name == winner:
    chosen = fitted
```

Phụ lục A Mã thí nghiệm phần 4

```
# Kiểm tra preprocessing chỉ fit trên Train.
np.testing.assert_allclose(chosen.named_steps["impute"].statistics_,
                           np.median(xtr, axis=0))
np.testing.assert_allclose(chosen.named_steps["scale"].mean_,
                           xtr.mean(axis=0))
# PCA day du là phép quay; Ridge không whitening phải tương đương.
direct = pipeline(Ridge(alpha=1, solver="svd")).fit(xtr, ytr)
full = pipeline(Ridge(alpha=1, solver="svd"), 10).fit(xtr, ytr)
rotation_error = float(np.max(np.abs(
    direct.predict(xte) - full.predict(xte))))
assert rotation_error < 1e-7
pca = full.named_steps["pca"]
variance = np.cumsum(pca.explained_variance_ratio_)
# Khao sát k riêng với alpha=1, không dùng để chọn lại winner.
ablation = []
for k in [2, 4, 6, 8, 10]:
    scores = []
    for tr, va in outer:
        fitted = pipeline(Ridge(alpha=1, solver="svd"), k)
        fitted.fit(xtr[tr], ytr[tr])
        scores.append(metrics(ytr[va], fitted.predict(xtr[va]))["RMSE"])
    ablation.append(dict(k=k, RMSE_mean=float(np.mean(scores)),
                        RMSE_std=float(np.std(scores, ddof=1))))
save_csv("pca_ablation.csv", ablation)
save_csv("test_metrics.csv", final_rows)
save_csv("test_predictions.csv", [dict(sample_id=int(i),
    target=float(yte[j]), **{n: v[j] for n, v in all_preds.items()})
    for j, i in enumerate(test)])
result = dict(winner=winner, cv=summary, test=final_rows,
    final_parameters=final_params, pca_variance=variance.tolist(),
    pca_ratio=pca.explained_variance_ratio_.tolist(),
    pca95_k=int(np.searchsorted(variance, 0.95) + 1),
    ablation=ablation, rotation_max_error=rotation_error,
    example=dict(sample_id=int(test[0]), x=xte[0].tolist(),
        target=float(yte[0]), prediction=all_preds[winner][0]),
    data=dict(n=442, p=10, n_train=353, n_test=89, missing=0,
        duplicate_X=int(len(x) - len(np.unique(x, axis=0))),
        target_range=[float(y.min()), float(y.max())],
        target_mean_train=float(ytr.mean()),
        feature_names=data.feature_names,
        sha256=hashlib.sha256(x.tobytes()+y.tobytes()).hexdigest()),
    versions=dict(python=platform.python_version()),
```

Phụ lục A Mã thí nghiệm phần 5

```
numpy=np.__version__, scipy=scipy.__version__,
sklearn=sklearn.__version__)
(OUT / "results.json").write_text(json.dumps(result, indent=2),
encoding="utf-8")
(OUT / "dataset_description.txt").write_text(data.DESCR, encoding="utf-8")
# Kiem tra metric doc lap bang NumPy cho mo hinh da chon.
e = np.asarray(all_preds[winner]) - yte
manual = dict(MAE=float(np.mean(abs(e))), RMSE=float(np.sqrt(np.mean(e**2))),
R2=float(1 - sum(e**2) / sum((yte-yte.mean())**2)))
observed = next(r for r in final_rows if r["model"] == winner)
assert all(abs(manual[m] - observed[m]) < 1e-10 for m in manual)
print("Selected before Test:", winner)
print("Test:", observed)
print("PCA 95% components:", result["pca95_k"])
print("All checks passed. Results:", OUT)

if __name__ == "__main__":
    main()
```